

08. Hidden Markov Models (HMM)

Algorithm implemented: Gaussian HMM trained with Baum-Welch (EM) and decoded with the Viterbi algorithm (hmmlearn).

Dataset: Financial market returns & volatility regimes (`data/market_regimes.csv`).

```
# ---- Core Libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from hmmlearn.hmm import GaussianHMM

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else
'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)

# ---- Load the market returns dataset ----
df_market = pd.read_csv('../data/market_regimes.csv')
print(f"Trading days: {len(df_market)}")
print(df_market.head(3))

returns = df_market['Daily_Return'].values.reshape(-1, 1) # HMM observations
prices = df_market['Close'].values                        # only for visualization
```

```
Trading days: 1000
   Date   Close  Daily_Return  True_Regime
0 2020-01-01  99.41     -0.00587           0
1 2020-01-02  99.68      0.00271           0
2 2020-01-03  99.93      0.00247           0
```

Gaussian HMM

Step-by-Step Algorithm (Gaussian HMM)

1. **Model:** a hidden 1st-order Markov chain Z with transition matrix A , emitting observations $x_t \sim N(\mu_k, \sigma_k^2)$ from the active state.
2. **Learning (Baum-Welch / EM):** iterate until convergence: a. **E-step:** run the forward-backward algorithm to compute posterior state probabilities $\gamma_t(k)$ and $\xi_t(i,j)$. b. **M-step:** re-estimate start probabilities, transition matrix A , and Gaussian parameters (μ_k, σ_k^2) from the posteriors.
3. **Decoding (Viterbi):** dynamic programming finds the single most likely state sequence: $\delta_t(j) = \max_i [\delta_{t-1}(i) A_{ij}] \cdot b_j(x_t)$; $\psi_t(j) = \operatorname{argmax}_i [\dots]$; backtrack from T to 1 .
4. **Evaluation:** the forward algorithm gives $P(X | \lambda)$, reported as the model log-likelihood.
5. **Post-processing:** sort states by volatility so regime labels (Bull/Sideways/Bear) are stable and interpretable.

```
# ---- Fit a 3-state Gaussian HMM with the Baum-Welch (EM) algorithm ----
n_components = 3
hmm_model = GaussianHMM(
    n_components=n_components,          # Bull / Sideways / Bear regimes
    covariance_type="full",             # each state has its own full covariance Gaussian emission
    n_iter=200,                         # EM iterations
    random_state=42,
    verbose=False
)
```

```

hmm_model.fit(returns)
print("EM Training Converged:", hmm_model.monitor_.converged)
print(f"Model Log-Likelihood: {hmm_model.score(returns):.2f}")

# ---- Decode the most likely hidden state sequence with the Viterbi algorithm ----
hidden_states = hmm_model.predict(returns)

# ---- Sort regimes by volatility so labels have a consistent economic meaning ----
# Regime 0: Low Volatility (Bull) | Regime 1: Moderate (Sideways) | Regime 2: High (Bear/Crisis)
volatilities = [np.sqrt(hmm_model.covars_[i][0][0]) for i in range(n_components)]
state_order = np.argsort(volatilities) # ascending volatility
state_mapping = {old_state: new_state for new_state, old_state in enumerate(state_order)}

sorted_hidden_states = np.array([state_mapping[s] for s in hidden_states]) # relabel sequence
sorted_means = [hmm_model.means_[old][0] for old in state_order] # reordered means
sorted_stds = [np.sqrt(hmm_model.covars_[old][0][0]) for old in state_order] # reordered std devs

# ---- Reorder the transition matrix to match the sorted regimes ----
sorted_transmat = np.zeros((n_components, n_components))
for i in range(n_components):
    for j in range(n_components):
        sorted_transmat[i, j] = hmm_model.transmat[state_order[i], state_order[j]]

regime_names = ["Bull (Low Vol)", "Sideways (Med Vol)", "Bear (High Vol)"]

```

```

EM Training Converged: True
Model Log-Likelihood: 3018.45

```

```

# ---- Transition probability matrix ----
fig, ax = plt.subplots(figsize=(6, 4.5))
sns.heatmap(sorted_transmat, annot=True, fmt='.3f', cmap='Blues', ax=ax,
            xticklabels=regime_names, yticklabels=regime_names, cbar=False)
ax.set_title("HMM Transition Probability Matrix ($A_{ij}$)", fontweight='bold')
ax.set_xlabel("Transition To Regime")
ax.set_ylabel("Current Regime")
plt.tight_layout()
plt.show()

# ---- Viterbi-decoded regimes over the price series ----
colors = ['forestgreen', 'royalblue', 'crimson']

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(13, 7), sharex=True, gridspec_kw={'height_ratios': [3, 1]})

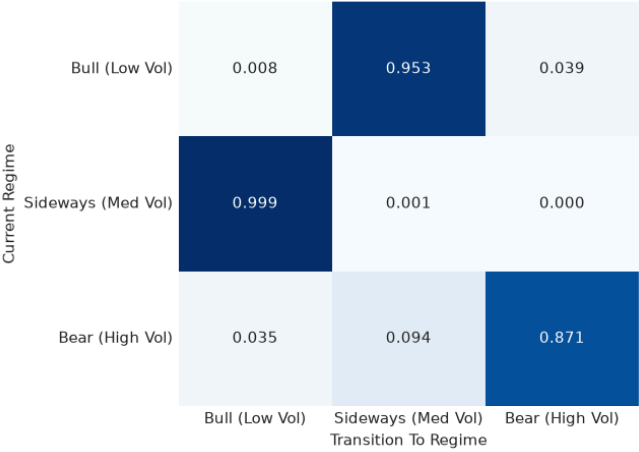
# Top: color every consecutive price segment by its decoded regime
for i in range(len(prices) - 1):
    ax1.plot([i, i + 1], [prices[i], prices[i + 1]], color=colors[sorted_hidden_states[i]], lw=1.8)
for i, name in enumerate(regime_names):
    ax1.plot([], [], color=colors[i], label=name, lw=2.5)
ax1.set_title("Asset Price Colored by Decoded HMM Regime (Viterbi Path)", fontsize=13,
fontweight='bold')
ax1.set_ylabel("Asset Price ($)")
ax1.legend(loc='upper left')

# Bottom: the raw sequence of decoded states over time
ax2.scatter(range(len(sorted_hidden_states)), sorted_hidden_states,
            c=[colors[s] for s in sorted_hidden_states], s=15, marker='|')
ax2.set_yticks(range(n_components))
ax2.set_yticklabels(regime_names)
ax2.set_xlabel("Trading Day Index")

plt.tight_layout()
plt.show()

```

HMM Transition Probability Matrix (A_{ij})



Asset Price Colored by Decoded HMM Regime (Viterbi Path)

